

Traversing the Twin Peaks

Mehdi Mirakhorli and Jane Cleland-Huang, DePaul University



IN INTERVIEWS CONDUCTED last October, this issue's guest editors, Mehdi Mirakhorli and Jane Cleland-Huang, spoke with Daniel Dvorak and Jan Bosch, who shared their perspectives as software architects on the interweaving of requirements and architecture. Dvorak is the lead for the NASA software architecture review board and has worked on various technology development projects and space missions, including Deep Space One and Mars Science Laboratory. Bosch is a software architect and agilest with a wealth of experience designing and implementing software-as-a-service and Web 2.0 solutions that typically provide services to hundreds of thousands, even millions, of customers.

Daniel Dvorak's Perspectives

Guest Editors: To provide some context for our readers, could you briefly describe the kinds of projects you work on at NASA and JPL?

Dan Dvorak: My first project was working on Deep Space One, a mission that demonstrated a dozen new technologies, including a set of software capabilities known as the "remote agent" that demonstrated a more model-based approach to automated spacecraft monitoring and control. It was billed as the first AI in space and won a NASA Software of the Year Award. That was neat. I've also worked on some technology development tasks, including one where we controlled a research rover in the JPL Mars Yard with software written in real-time Java. That project was joint with Sun Microsystems, and I got to present a summary of it on stage with James Gosling at the huge Java One conference. Biggest audience I ever addressed!

GEs: What kind of development process do you use? How much interplay happens between requirements and architecture?

Dvorak: There's a lot of back and forth between requirements and architecture.

In our space missions, scientists specify ambitious science objectives that have to be tempered by cost and risk as well as engineering constraints. Also, I want to emphasize that architecting is not something that's done once early in a project; there's an interplay between requirements and architecture at each level of refinement, from system level to subsystem level to component level.

Some people in the past have tried to follow a waterfall approach. In practice, it's much more iterative than that, and sometimes you might go down a certain road believing you have the right set of requirements and you realize there's an engineering constraint and you can't do something as fully as you would like. There's always going to be some back-and-forth interplay. There does tend to be an iterative approach in the development of functionality and testing. The length of the iteration cycles varies from a few weeks to a few months depending on the project.

GEs: One of the concepts that people are talking about a lot recently is the concept of an architecturally significant requirement. How would you define such a requirement?

Dvorak: In our software architecture reviews, we make a distinction between driving requirements and key requirements. Driving requirements are architecturally significant requirements in the sense that they tend to drive or shape an architecture, usually in the form of changes to an existing architecture from a previous project. There might be many other requirements that are just as important, but if they can be satisfied using mature, flight-proven areas of the existing architecture, then they aren't driving us into new territory. In an architecture review, we



focus more attention on driving requirements because they entail more risk. Key requirements, by the way, are the requirements that are most important to stakeholders, so it's an orthogonal classification; a driving requirement may or may not be key, and vice versa.

GEs: What's your process for eliciting and specifying requirements? Do you work closely with requirements engineers?

Dvorak: Let me preface my answer by pointing out that spacecraft are complex cyber-physical systems, and there are systems engineers of various specialties who work together to define requirements from the system level on down to subsystem and component levels, and it's the software systems engineers who are responsible for the software requirements. There are requirements on both the flight system and the ground system since these two elements need to work together as a system. I view requirements as the end result of an architecting process. If you think of architecting in the sense of ISO 42010, you have to identify the stakeholders and their concerns, and define viewpoints on the system that addresses those concerns. Architecting is the process of trying to reconcile the many different competing concerns, so there's always some iteration going on.

GEs: In a typical project, what kinds of iterations occur between requirements and architecture design activities?

Dvorak: Much of NASA's work involves first-of-a-kind or one-of-a-kind missions, so there's a lot of uncertainty about what's feasible within a given cost and schedule. Also, as a system architecture takes shape, we see myriad

physical effects that have to be managed. For example, a spacecraft's batteries might not be as big as desired because larger batteries have more mass, making it more expensive to launch. That, in turn, means that power consumption must be carefully managed,

that it was overly stringent—quickly relaxed the requirement. Unfortunately, the damage was already done; an unsubstantiated requirement had spawned an unnecessary cascade of time-consuming analysis, design, development, and testing, not to mention the

Recipients of a hard-to-satisfy requirement shouldn't suffer in silence.

including taking time to recharge the batteries. It's not too much of an exaggeration to say that "everything affects everything" on our science spacecraft, and software is in the middle of managing those interactions. That's why there's so much iteration between requirements and architecture.

GEs: Can you think of any situations in which architecturally significant requirements were identified late in the project, after the architecture was designed?

Dvorak: An example surfaced in the NASA study of flight software complexity [www.nasa.gov/offices/oce/documents/FSWC_study.html]. In one mission, a scientist levied a requirement for "99 percent data completeness," implying that science results would be greatly diminished by any interruptions in science observations. The flight software team took the requirement seriously and designed and developed a system with redundant elements and fast onboard fault detection and response, making for a more complex system. Later in the project, somebody questioned the value of 99 percent, and the scientist—realizing

dismay of the software team who had worked so hard. There are two lessons in this example. First, some requirements, especially performance requirements, should be substantiated with a documented rationale that is subjected to scrutiny. Second, recipients of a hard-to-satisfy requirement shouldn't suffer in silence; requirements are rarely absolute, and they should inform project management about the impact of a difficult requirement on schedule and cost and technical risk.

GEs: Let's talk about the role of reference models for architecting. How much do you rely on reusing existing architectures?

Dvorak: Projects commonly reuse existing architectures in an effort to save cost and reduce risk. For example, the MSL rover that recently landed on Mars [2012] traces its software architecture roots back to the MER rover [2003] and further back to the Mars *Pathfinder* rover [1996]. In a similar vein, NASA/GSFC developed a reference architecture and core software, known as "core flight executive/core flight software" [cFE/CFS; [MARCH/APRIL 2013 | IEEE SOFTWARE 31](http://</p></div><div data-bbox=)

flightsoftware.jhuapl.edu/files/2011/FSW11_Cudmore.pdf], specifically to support a class of science missions. The Goddard team made an observation that bears repeating: “It was the software architecture that lived on, even as hardware came and went.” In other words, even though each mission is different, there’s a level of abstraction captured in a reference architecture that provides useful structure that shapes the detailed design.

Before I leave this topic, it’s important to note a pitfall in reusing an existing architecture, and that is that it may not scale or extend well to satisfy a new project’s driving requirements. For the people who are accustomed to working within an existing architecture, it might be hard to recognize a looming problem and decide that a new approach is needed. This is the kind of situation where a non-advocate review by a diverse team of architects can help. That’s why architecture reviews can be so helpful if performed when a project is in the formative stages of

architecting. In the NASA project life cycle, that time is well before PDR [preliminary design review].

GEs: Given that you reuse solutions from previous missions, do you experience situations in which new requirements for a mission are constrained by the architecture already in place?

Dvorak: This is an area where the answer is changing from “rarely” to “commonly,” so let me explain why. When I first arrived, JPL was in a mode of doing one big flagship mission every 10 years, so we tended to start from a blank sheet of paper each time and not feel constrained much by older architectures. However, today, we have multiple projects in development at the same time, and it’s just not cost-effective to start from scratch on each one and create the “perfect” architecture. That’s why we’re moving toward reference architectures that will accommodate certain variations readily, but not all.

Thus, some requirements will be constrained by the architecture.

GEs: As we’re focusing on the interplay between requirements and architecture, we’re interested in whether you have any techniques in place for preserving these relationships. For example, do you capture design rationales?

Dvorak: In a lot of cases in the past, those trade-offs have not been captured. However, we’re moving toward model-based systems engineering and are advocating for capturing the various analyses that lead you to decide one thing versus another. For example, I might have a couple of requirements at one level and need to do an analysis based on those requirements and maybe other aspects of the design to decide the next lower-level requirement. I want that analysis to be captured and preserved for somebody in the future who might join this project later in the life cycle, or somebody in the future who has a new mission and is looking at the old mission to understand what was decided and why. This is an area in which we’re trying to improve.

GEs: What about traceability? Do you have traceability in place between requirements and key architectural decisions?

Dvorak: There’s quite a strong emphasis on traceability between requirements and decisions. You mentioned traceability among key decisions—that’s one of the things that the software architecture review board emphasizes in architectural description documents. We want to see the key architectural decisions and why they were made, and also to understand any underlying assumptions.

GEs: Do you trace architectural decisions forward into the implementation, too, to make sure that the implemented system



is correct with respect to the requirements or to help developers understand the underlying architectural decisions?

Dvorak: One lesson we've learned about programming is that having well-documented coding rules doesn't ensure that they'll be consistently followed. It's important to have tools that check the code for compliance. By analogy, it's important to have tools that check the design for compliance with the architecture. Otherwise, they will diverge, conceptual integrity will be lost, and the resulting system will be far less than what the architecture promised. Also, many software-intensive systems will have lifetimes measured in decades and will be modified by people who never met the original architects. It's vitally important that architects clearly document the rationale for key architectural decisions so that future maintainers/architects understand the impacts of extending or changing the architecture.

GEs: Thank you for your candid insights. Do you have any closing comments you would like to share?

Dvorak: It's useful to step back once in a while and look at history. In 1968, NATO held the first conference on software engineering to address the "software crisis," and the problems that existed then are still familiar today: poor performance, inadequate reliability, schedule overruns, poor cost estimation, unmet requirements, and education of software engineers. What's different, though, is that advances since 1968 have enabled software systems three orders of magnitude larger while software cost as a fraction of total budget has remained small. What was once considered complex became routine because we learned better ways, and the emergence of software architecture as a discipline has been key.

Jan Bosch's Perspectives

Guest Editors: Can you tell us something about the kinds of systems you've been working on?

Jan Bosch: I've worked on anything from embedded systems and safety-critical systems to software-as-a-service and Web 2.0 solutions. The commonality, however, is that these systems have large markets of customers, often consumers but also businesses, and the products are typically products and services provided to the hundreds thousands or even millions.

GEs: As you know, our interest is in the interplay between requirements and architecture, which seems to hinge on the notion of an architecturally significant requirement. Perhaps we could start by asking you how you would define an architecturally significant requirement?

Bosch: Well, if you go back to when we started to look at architecturally significant requirements in the 1990s, we

were basically looking for the requirements in which we make architecture design decisions that would be really hard to change after the system's initial development. That perspective is still valid, that there is a subset of requirements that are architecturally significant. We have to identify and capture those requirements and take them into the architectural design early on. That's the traditional way of thinking about it.

GEs: In your projects, who's actually responsible for eliciting and specifying requirements?

Bosch: In traditional companies, the product managers bring together the needs of all the stakeholders. However, what I now see is that architects and lead developers are increasingly being added to that mix and are also involved with discussions with project stakeholders.

GEs: To what extent do you document requirements, especially in cases where



they're gathered by product managers and passed along to architects and developers?

Bosch: I'm of the opinion that it's less and less about documents and document models and things like that, and more and more about getting architects, engineers, and others to meet with customers to build an empathy and understanding for customer needs. This is a change because we traditionally didn't think of this as a responsibility of architects and engineers. In the company I previously worked for, we had a practice known as "follow me home," which meant that everyone who was active in product development in the company would visit a customer at least once a year for a day, to spend time with the customer to understand what the customer needs. I believe this is a much more effective approach for understanding the requirements than writing them down in documents.

But I'm not saying there's no value in documents; it has just proven in my case to be much more effective to have architects and people responsible for specific components to have that tacit knowledge in their heads, and then to share it with people in more of a coaching, mentoring style whenever the need

knowledge, and what kind of models or formats would you favor for documenting that knowledge?

Bosch: It's not that I'm against documentation, but rather that I see so little use of it in industrial practice. The most useful forms of documentation are views of the software that can be automatically generated. Everything that's disconnected from the assets that deliver the key value to customers tends to get outdated very quickly and consequently is considered useless. For other types of documentation, I mostly see slide presentations capturing the high-level structure and rationale. These are often effective as an introduction. Understanding more often requires getting into the code.

GEs: In traditional projects, architecturally significant requirements tend to be dealt with up front, to manage some of the project risks, but can you say something about how you make requirements prioritization decisions in an agile project?

Bosch: Most of the companies that I've worked with tend to start from the traditional perspective; however, I see that people are switching from quality-

cost of doing business. As a community, we're starting to accept that we have to invest in refactoring continuously.

The whole notion of starting from a predefined requirements specification that customers have given to us in some magical shape, way, or form is a fallacy. It's an illusion, a myth. We can deduce some requirements based on what the customer has told us, and there are other things that we can't know until we test and experiment with them. So while I think there's a place for requirements that we just know a system must support, there's also a place for experimenting and trying things out and getting runtime customer feedback rather than assuming that the customer knows exactly what he or she wants from us in the first place. For example, in the Web 2.0 space, companies are constantly running experiments in their live product to understand what's important for customers. So, in this respect, the real requirements were learned through customer feedback.

GEs: Have you experienced any cases in which architecturally significant requirements were identified after the architecture design?

Bosch: Yes. Although the traditional understanding is that it's very costly and difficult to fix the architecture after the fact, in the more agile development context, this happens all the time. People request new features, which calls for refactoring, and except for the really, really big refactoring efforts that just hurt everything, people take all this in stride and view at it as par for the course. Occasionally, we have a situation where the team first built one solution that they felt was going to be really solid and successful, but later on when the system evolved and requirements changed, they realized that the current architecture wasn't suitable, and they basically paid the price in terms

Many software-intensive systems will have lifetimes measured in decades.

arises to make changes to the existing architecture. So you have people who know the history of the system and can inform others.

GEs: Under what circumstances, if any, would you consider it helpful to document requirements or architectural

based architecture work to user-driven architecture work. This means that the team tries to deliver requirements with the highest customer value first. That's also the reason that the community has become more accepting of the fact that sometimes you have an architecture that isn't ideal and refactoring is just a

of taking a significant chunk of R&D effort to reengineer the architecture to cover the system needs. If we look at the domains that aren't safety critical or where there is a significant embedded systems component, then we often find the majority of architecturally significant requirements after system development and sometimes even after system maintenance has been started.

GEs: Does the existing architecture constrain the viability of accepting new requirements?

Bosch: It becomes purely an economic question, and I have to decide whether I'm willing to implement a feature at a higher cost than I would have done normally, making it an expensive feature implementation. If we ask whether it's "more or less difficult" to implement certain requirements because of the architecture in place, the answer is absolutely "Yes!" I've seen several such cases in my consulting engagement as well as in the companies that I've worked for. However, saying that existing architectural constraints make it impossible to introduce a new requirement is a bridge too far. I've never had a case where the current architecture prevented us from building something that we really wanted. We have to decide whether to build a feature within the constraints of the existing architecture or whether to invest in refactoring the architecture and then implementing the feature at a lower cost. I expect every requirement that I build to provide a certain amount of value to the user community, and if the architecture makes the cost of implementing the requirement so high that it exceeds the value that the user community will get from that requirement, then of course we're in a situation where the architecture constrains the requirement because there's no

longer a business case for implementing that requirement.

GEs: One way in which people manage incremental development is through the use of proven reference architectures. Do you see the use of existing solutions as a way to address some of the uncertainty that goes into the development effort?

The most useful forms of documentation are views of the software that can be automatically generated.

Bosch: That's a really good question. If we talk about reference architectures or standard architectures that exist within a community for developing a Web server or creating a build system, I see that there's a lot of sharing and a lot of starting from reference architectures. I also have done quite a bit of work in software product lines, and I'm surprised at how little reuse of design principles, design ideas, and software assets takes place within companies. Some are very good at it and take a strong platform approach, but I still run into companies where different teams are just running around doing their own thing without learning from others in the same organization.

GEs: That's interesting because we often hear that agile approaches are more effective when there are known architectural solutions, perhaps in the form of reference architectures. But are you saying that many of the projects you're involved in develop architectures incrementally, even without the use of reference architectures?

Bosch: Once you realize that virtually all software development is concerned with evolving existing software and

that starting from a green field is the rare exception, I think the perspective changes. Most professionals work with evolving, existing implemented architectures, and our focus should be on helping them do that in the best possible way. Whenever a system needs to be retired because of architectural erosion, it isn't a natural cause of aging but the

consequence of underinvesting in refactoring over the product's life cycle. As long as a software system has users, it shouldn't need to retire.

GEs: Let's move on and talk about the relationships between requirements and architecture during longer-term maintenance efforts. You aren't in favor of documentation, but do you see any role for traceability in your projects, especially for the purpose of capturing relationships between requirements, architecture, and the implementation of architectural decisions in source code?

Bosch: Here again it's less about documented traceability. I'm increasingly hesitant about using documents per se and increasingly focused on the role of the architect or architects in this context. In this case, I believe it's the job of the architect to understand the architectural design decisions that have been taken in response to specific requirements and how those decisions influence the implementations that the engineers do. An architect is not just a PowerPoint pusher; in most of the companies I've worked in, the architects spend between 30 and 70 percent of their time coding, but they also



DANIEL DVORAK is a principal engineer in the Systems and Software Division at the Jet Propulsion Laboratory, California Institute of Technology. He also serves as division technologist, supports an initiative on model-centric engineering, and leads NASA's software architecture review board. Dvorak's interests include model-based systems engineering, fault management, robot control architectures, and goal-driven mission operations. He received a PhD in computer science from the University of Texas at Austin. Contact him at daniel.i.dvorak@jpl.nasa.gov.



JAN BOSCH is a professor of software engineering and director of the software research center at Chalmers University of Technology in Gothenburg, Sweden. Previous to this role, he worked as vice president of engineering process at Intuit, where he led Intuit's open innovation efforts and headed the central mobile technologies team. Bosch's research activities include open innovation, innovation experiment systems, compositional software engineering, software ecosystems, software architecture, software product families, and software variability management. He received a PhD in computer science from Lund University, Sweden. Contact him via www.janbosch.com.

spend part of their time on architecture and thinking about architecture.

GEs: How does that work over the long term, though? For example, what happens if people leave the project or the corporate memory of the product fades over time?

Bosch: In my experience, in many companies, the whole notion of product development has shifted from “this is a thing I do once” to a mode where the system is constantly evolving and there's a team that constantly invests in product development. When you do that, you can rely a lot more on individual knowledge, but if you have a situation where these things are much more driven in a project fashion, where there's work being done and then everyone goes away again to work on something else, then maintaining the knowledge becomes much more important.

The main point that I'm trying to get across is that the time in which engineers and architects would get the requirement specification from customers and then would build the system according to the specification is gone. Instead, architects and engineers need to be out there, meeting customers and understanding how their system will be used, getting an MVP [minimal viable product] in the hands of these customers and then evolving the system in short iterations while constantly observing and measuring use and using that data to steer development. ☞



**Richard E. Merwin
Student Leadership
Scholarship**

IEEE Computer Society is offering \$40,000 in student scholarships, from \$1,000 and up, to recognize and reward active student volunteer leaders who show promise in their academic and professional efforts.

Graduate students and undergraduate students in their final two years, enrolled in a program in electrical or computer engineering, computer science, information technology, or a well-defined computer-related field, are eligible. IEEE Computer Society student membership is required.

Apply now! Application deadline is 30 April 2013. For more information, go to www.computer.org/scholarships, or email chuffman@computer.org.

To join IEEE Computer Society, visit www.computer.org/membership.

IEEE  computer society

 IEEE



Selected CS articles and columns are also available for free at <http://ComputingNow.computer.org>.